

# Cây cơ bản & DFS trên cây

## Nội dung

Cây (tree) là cấu trúc đồ thị đặc biệt: liên thông và không có chu trình. Cây có  $n$  đỉnh và đúng  $n - 1$  cạnh. Rất nhiều bài CP "ẩn cây" trong đề.

Trong bài này ta học các kỹ thuật DFS cốt lõi trên cây.

1. Tính chất cơ bản của cây
2. DFS trên cây — không cần visited
3. Tính kích thước cây con (subtree size)
4. Tính độ sâu (depth) — dòng chảy "cha  $\rightarrow$  con"
  - So sánh hai kiểu dòng chảy
  - Code
  - Ứng dụng phổ biến của pre-order
5. Khoảng cách BFS từ một đỉnh
6. Đường kính cây (Tree Diameter)
7. Trọng tâm cây (Centroid)
8. Dùng DFS post-order để gộp thông tin từ con
  - Ví dụ: Tổng khoảng cách từ  $u$  đến mọi đỉnh trong cây con
9. Stack overflow trên cây sâu
10. Bài toán điển hình
11. Tổng kết

## 1. Tính chất cơ bản của cây

Cây có gốc (rooted tree): chọn 1 đỉnh làm gốc  $\rightarrow$  mỗi đỉnh có 0 hoặc 1 cha,  $\geq 0$  con.

- $n$  đỉnh,  $n - 1$  cạnh.
- Giữa mọi cặp đỉnh có đúng 1 đường đi đơn giản.
- Bỏ 1 cạnh  $\rightarrow$  cây bị tách thành 2 thành phần.
- Thêm 1 cạnh  $\rightarrow$  tạo đúng 1 chu trình.

## 2. DFS trên cây — không cần visited

Trên cây có gốc, DFS chỉ cần truyền cha để tránh đi ngược:

C++

```
vector<int> g[N];
void dfs(int u, int par) {
    for (int v : g[u]) {
        if (v == par) continue;
        dfs(v, u);
    }
}
dfs(root, 0); // 0 = không có cha
```

## Python

```
import sys
sys.setrecursionlimit(300000)
def dfs(u, par):
    for v in g[u]:
        if v == par:
            continue
        dfs(v, u)
dfs(root, 0)
```

Đây là khung cho mọi DFS trên cây.

**Cảnh báo Python:** đệ quy mặc định chỉ  $\sim 1000$  mức. Cây có  $n \geq 10^5$  phải tăng `setrecursionlimit`. Cây sâu cực đại ( $n = 10^6$ ) vẫn có thể stack overflow — chuyển DFS sang vòng lặp với stack hoặc dùng C++.

## 3. Tính kích thước cây con (subtree size)

Trên cây có gốc,  $sz[u]$  = số đỉnh trong cây con gốc  $u$ . Tính lá trước ( $sz = 1$ ), rồi gộp giá trị con lên cha:

$$sz[u] = 1 + \sum_{v \text{ là con của } u} sz[v]$$

Mũi tên cam trong hình minh họa luồng giá trị đi từ con LÊN cha — đây là khung tư duy chung cho mọi DP trên cây kiểu post-order.

## C++

```
int sz[N];
void dfs_size(int u, int par) {
    sz[u] = 1;
    for (int v : g[u]) {
        if (v == par) continue;
        dfs_size(v, u);
        sz[u] += sz[v];
    }
}
```

## Python

```
sz = [0] * (n + 1)
def dfs_size(u, par):
    sz[u] = 1
    for v in g[u]:
        if v == par:
            continue
        dfs_size(v, u)
    sz[u] += sz[v]
```

$sz[u]$  = số đỉnh trong cây con gốc  $u$ .

**Ứng dụng:** bài toán đếm cặp  $(u, v)$  sao cho ... thường liên quan tới subtree size.

## 4. Tính độ sâu (depth) — dòng chảy "cha $\rightarrow$ con"

Bài toán tính  $sz[u]$  ở mục 3 đã minh họa một kiểu DP trên cây: con tính trước, cha gộp lại sau (post-order, "con  $\rightarrow$  cha"). Nhưng có một kiểu đối lập cũng cực kỳ quan trọng — cha tính trước, truyền giá trị xuống con (pre-order, "cha  $\rightarrow$  con"). Bài toán mẫu kinh điển: tính độ sâu  $dep[u]$  = số cạnh từ gốc tới  $u$ .

Truy hồi:  $dep[\text{gốc}] = 0$ ;  $dep[v] = dep[u] + 1$  với  $v$  là con của  $u$ .

### So sánh hai kiểu dòng chảy

|                  | <b>POST-ORDER</b> (CON $\rightarrow$ CHA) | <b>PRE-ORDER</b> (CHA $\rightarrow$ CON)     |
|------------------|-------------------------------------------|----------------------------------------------|
| Ví dụ            | $sz[u]$ , đường kính, tree DP             | $dep[u]$ , khoảng cách từ gốc, propagate màu |
| Tính ở đâu?      | Sau khi gọi đệ quy con                    | Trước khi gọi đệ quy con                     |
| Phụ thuộc        | $dp[u]$ phụ thuộc $dp[\text{các con}]$    | $dp[v]$ phụ thuộc $dp[\text{cha của } v]$    |
| Trình tự ghi nhớ | Lá xong trước, gốc xong sau               | Gốc xong trước, lá xong sau                  |

Đây là mẫu khung quan trọng: khi gặp bài DP trên cây, việc đầu tiên là hỏi — "giá trị của  $u$  phụ thuộc cha hay phụ thuộc con?" Câu trả lời quyết định chỗ đặt câu lệnh tính toán.

## Code

### C++

```
int dep[N];
void dfs_dep(int u, int par) {
    for (int v : g[u]) {
        if (v == par) continue;
        dep[v] = dep[u] + 1; // TÍNH dep[v] TRƯỚC khi đệ quy
        dfs_dep(v, u);
    }
}
```

```
dep[root] = 0;
dfs_dep(root, 0);
```

## Python

```
dep = [0] * (n + 1)
def dfs_dep(u, par):
    for v in g[u]:
        if v == par:
            continue
        dep[v] = dep[u] + 1 # TÍNH dep[v] TRƯỚC khi đệ quy
        dfs_dep(v, u)
dep[root] = 0
dfs_dep(root, 0)
```

So sánh với code tính  $sz[u]$  ở mục 3: dòng  $sz[u] += sz[v]$  ở sau lời gọi  $dfs\_size(v, u)$  (vì cần  $sz[v]$  đã được tính). Còn ở đây dòng  $dep[v] = dep[u] + 1$  đặt trước lời gọi  $dfs\_dep(v, u)$  (vì cần  $dep[u]$  đã có sẵn).

## Ứng dụng phổ biến của pre-order

- Khoảng cách từ gốc (số cạnh hoặc tổng trọng số) — mở rộng **dep** thành tổng.
- Truyền màu / nhãn từ cha xuống con (ví dụ tô màu thay phiên 0/1 cho cây bipartite).
- Đánh số DFS pre-order (in-time, dùng cho Euler tour ở khóa Vàng).

## 5. Khoảng cách BFS từ một đỉnh

Trên cây không trọng số, BFS từ  $s \rightarrow$  khoảng cách  $d[v]$  = số cạnh ngắn nhất từ  $s$  đến  $v$ . Tương đương DFS.

## 6. Đường kính cây (Tree Diameter)

Đường kính = đường đi dài nhất trong cây.

### Thuật toán 2 lần BFS/DFS:

1. BFS từ một đỉnh bất kỳ  $s \rightarrow$  tìm đỉnh xa nhất  $u$ .
2. BFS từ  $u \rightarrow$  đỉnh xa nhất  $v$ .  $dist(u, v)$  chính là đường kính.

(Trong hình minh họa: bắt đầu BFS từ một đỉnh bất kỳ  $s$  tìm được đỉnh xa nhất là  $u = 4$ . Sau đó BFS từ  $u$  tìm được đỉnh xa nhất là  $v = 9$ . Đường  $4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 9$  dài 7 cạnh chính là đường kính.)

C++

```
// BFS từ s, trả (đỉnh xa nhất, khoảng cách)
pair<int, int> far(int s) {
    vector<int> d(n+1, -1);
    queue<int> q; q.push(s); d[s] = 0;
    int best = s;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (d[u] > d[best]) best = u;
        for (int v : g[u]) if (d[v] == -1) { d[v] = d[u] + 1; q.push(v); }
    }
    return {best, d[best]};
}
auto [u, du] = far(1); // Lần 1: từ một đỉnh bất kỳ
auto [v, diameter] = far(u); // Lần 2: từ u -> ra đường kính
```

## Python

```
from collections import deque
def far(s):
    d = [-1] * (n + 1)
    d[s] = 0
    q = deque([s])
    best = s
    while q:
        u = q.popleft()
        if d[u] > d[best]:
            best = u
        for v in g[u]:
            if d[v] == -1:
                d[v] = d[u] + 1
                q.append(v)
    return best, d[best]
u, _ = far(1)
v, diameter = far(u)
```

## 7. Trọng tâm cây (Centroid)

**Định nghĩa:** Trọng tâm là đỉnh  $u$  sao cho khi xóa  $u$ , mỗi cây con còn lại có không quá  $n/2$  đỉnh.

**Tính chất:** Mọi cây đều có ít nhất một trọng tâm (có thể có 2 khi  $n$  chẵn).

**Thuật toán:** dùng  $sz[]$  từ mục 3. Một đỉnh  $u$  là trọng tâm khi:

- Mọi cây con là con của  $u$  có kích thước  $\leq n/2$ , và
- Phần còn lại "phía cha" có kích thước  $n - sz[u] \leq n/2$ .

( $par[u]$  = cha của  $u$  sau khi DFS; với gốc,  $par[gốc] = 0$  và "phía cha- 0 đỉnh.) Centroid xuất hiện trong nhiều thuật toán cây nâng cao (centroid decomposition — khóa Vàng).

C++

```
int findCentroid() {
    for (int u = 1; u <= n; u++) {
        bool ok = (n - sz[u] <= n / 2);
        for (int v : g[u]) {
            if (v != par[u] && sz[v] > n / 2) { ok = false; break; }
        }
        if (ok) return u;
    }
    return -1;
}
```

Python

```
def find_centroid():
    for u in range(1, n + 1):
        ok = (n - sz[u] <= n // 2)
        for v in g[u]:
            if v != par[u] and sz[v] > n // 2:
                ok = False
                break
        if ok:
            return u
    return -1
```

## 8. Dùng DFS post-order để gộp thông tin từ con

Mẫu: tính  $dp[u]$  (hoặc  $f(u)$ ) phụ thuộc vào  $dp[v]$  của các con.

C++

```
long long dp[N];
void dfs(int u, int par) {
    dp[u] = 0; // Hoặc giá trị khởi tạo phù hợp
    for (int v : g[u]) {
        if (v == par) continue;
        dfs(v, u);
        dp[u] += dp[v]; // Hoặc max, hoặc công thức khác
    }
    dp[u]++; // Hoặc +giá trị đỉnh u
}
```

Python

```
dp = [0] * (n + 1)
def dfs(u, par):
    dp[u] = 0
    for v in g[u]:
```

```

        if v == par:
            continue
        dfs(v, u)
        dp[u] += dp[v]
    dp[u] += 1

```

**Ví dụ: Tổng khoảng cách từ  $u$  đến mọi đỉnh trong cây con**  
C++

```

long long sum_dist[N], sz[N];
void dfs(int u, int par) {
    sz[u] = 1; sum_dist[u] = 0;
    for (int v : g[u]) {
        if (v == par) continue;
        dfs(v, u);
        sz[u] += sz[v];
        sum_dist[u] += sum_dist[v] + sz[v];
        // Mỗi đỉnh trong subtree(v) cách u thêm 1 so với cách v
    }
}

```

**Python**

```

sz = [0] * (n + 1)
sum_dist = [0] * (n + 1)
def dfs(u, par):
    sz[u] = 1
    sum_dist[u] = 0
    for v in g[u]:
        if v == par:
            continue
        dfs(v, u)
        sz[u] += sz[v]
        sum_dist[u] += sum_dist[v] + sz[v]

```

## 9. Stack overflow trên cây sâu

Cây có thể sâu tới  $n \rightarrow$  DFS đệ quy với  $n = 10^6$  có thể stack overflow.

**Giải pháp:**

- Tăng stack: `ulimit -s unlimited` (một số Online Judge hỗ trợ).
- Viết DFS không đệ quy bằng stack.
- Trên hầu hết Online Judge,  $n \leq 2 \times 10^5$  thường an toàn.

## 10. Bài toán điển hình

- Đường kính cây.
- LCA cơ bản — đi lên cha cho tới khi gặp.
- Đếm số con của mỗi đỉnh / kích thước cây con.
- Đếm cặp đỉnh có khoảng cách đúng  $k$ .
- Tâm cây / Trọng tâm cây.
- Tổng độ dài tất cả đường đi trên cây.

## 11. Tổng kết

Cây là cấu trúc đặc biệt của đồ thị — DFS trên cây luôn:

1. Truyền cha để khởi đi ngược.
2. Post-order: gộp thông tin từ con.
3. Pre-order: truyền thông tin từ cha xuống. enumerate

Học vững cây là tiền đề cho DP trên cây (Vàng), LCA, HLD, centroid decomposition, ... — toàn những thuật toán đắt giá!